Práctica 06: Gestión de Recursos, Patrón Flyweight y Fugas de Memoria

Alumno: Elliot Kenneth Villafaña Pasten

Asignatura: Fundamentos de Motores de Videojuegos con C++

Lenguaje: C++20

Bibliotecas: SDL3 y SDL3_image

Entorno de desarrollo: Windows 11, MSYS2 UCRT64 y CMake

Repositorio: https://github.com/Piabonnie-Dev/practica02-motor

Etiqueta de entrega: v0.6

1. Objetivo

Implementar un administrador de recursos gráficos (`ResourceManager`) utilizando el patrón de diseño Flyweight y la técnica RAII, con el propósito de evitar la carga innecesaria de texturas y administrar correctamente la memoria.

También se desarrolló un componente `SpriteComponent` para renderizar imágenes PNG y una escena de demostración que permite comprobar cómo múltiples entidades pueden compartir una misma textura.

2. Desarrollo e implementación

Para esta práctica retomé la arquitectura de componentes que había desarrollado anteriormente en mi proyecto Space Invaders.

Implementé la clase `ResourceManager`, que utiliza un `std::unordered_map` para almacenar las texturas mediante la ruta de sus archivos. Cuando se solicita una imagen, el método `GetTexture()` primero comprueba si ya está almacenada en la caché. Si existe, devuelve su puntero; de lo contrario, carga la imagen y la almacena para reutilizarla.

Las texturas se administran mediante `std::unique_ptr` y un destructor personalizado llamado `TextureDeleter`, que ejecuta `SDL_DestroyTexture()` al liberar los recursos.

También implementé `SpriteComponent`, que obtiene la posición y escala desde `TransformComponent` y dibuja las imágenes utilizando `SDL_RenderTextureRotated()`.

Este componente permite aplicar modificaciones de color y transparencia. Después de renderizar, restaura los valores predeterminados para evitar que las modificaciones visuales afecten a otras entidades que compartan la textura.

3. Aplicación del patrón Flyweight

El patrón Flyweight permite compartir información común entre diferentes objetos para reducir el consumo de memoria.

En mi implementación se distinguen dos tipos de estado:

Estado intrínseco: corresponde a los datos compartidos, principalmente la textura almacenada y administrada por `ResourceManager`.

Estado extrínseco: corresponde a las propiedades individuales de cada entidad, como su posición, escala, color y transparencia.

La escena `ResourceDemoScene` crea inicialmente 52 entidades: un fondo, una nave y 50 enemigos. Sin embargo, solamente necesita tres texturas distintas:

- `background.png`
- `player.png`
- `enemy.png`

Los 50 enemigos comparten la textura `enemy.png`, aunque cada uno tiene su propio `GameObject`, `TransformComponent` y `SpriteComponent`.

Al presionar ESPACIO, el programa genera ocho enemigos adicionales y consulta nuevamente la textura compartida, sin necesidad de cargar otra copia de la imagen.

De esta manera se demuestra que el número de entidades puede aumentar mientras la cantidad de texturas almacenadas permanece constante.

4. Ciclo de vida de las imágenes y administración de memoria

Para cargar una imagen se utilizan dos estructuras principales:

SDL_Surface: representa los píxeles de la imagen decodificada en la memoria RAM de la CPU.

SDL_Texture: representa una textura utilizada por el sistema de renderizado, normalmente almacenada en memoria gráfica o VRAM.

El proceso de carga implementado es el siguiente:

1. `IMG_Load()` lee el archivo PNG y genera una superficie en RAM.
2. `SDL_CreateTextureFromSurface()` crea una textura a partir de esa superficie.
3. `SDL_DestroySurface()` libera la memoria utilizada por la superficie.
4. La textura se almacena en `ResourceManager` mediante `std::unique_ptr`.
5. Cuando se limpia la caché, el destructor personalizado libera la textura mediante `SDL_DestroyTexture()`.

Es importante ejecutar `SDL_DestroySurface()` después de crear la textura, ya que los píxeles de la superficie dejan de ser necesarios. Omitir esta instrucción provocaría que permaneciera memoria reservada sin liberar.

La técnica RAII permite que la liberación de las texturas esté asociada al ciclo de vida de los objetos que las administran, disminuyendo el riesgo de fugas de memoria.

5. Análisis de fugas de memoria y AddressSanitizer

AddressSanitizer es una herramienta que permite identificar errores relacionados con el manejo de memoria, como accesos inválidos y, cuando está disponible LeakSanitizer, fugas de memoria.

Para demostrar este problema se implementó `TriggerDeliberateLeak()`, un método que se activa al presionar la tecla L.

Esta función ejecuta `IMG_Load()` repetidamente para crear 50 superficies, pero omite intencionalmente `SDL_DestroySurface()`.

Al no liberar estas superficies, se genera una fuga de memoria deliberada.

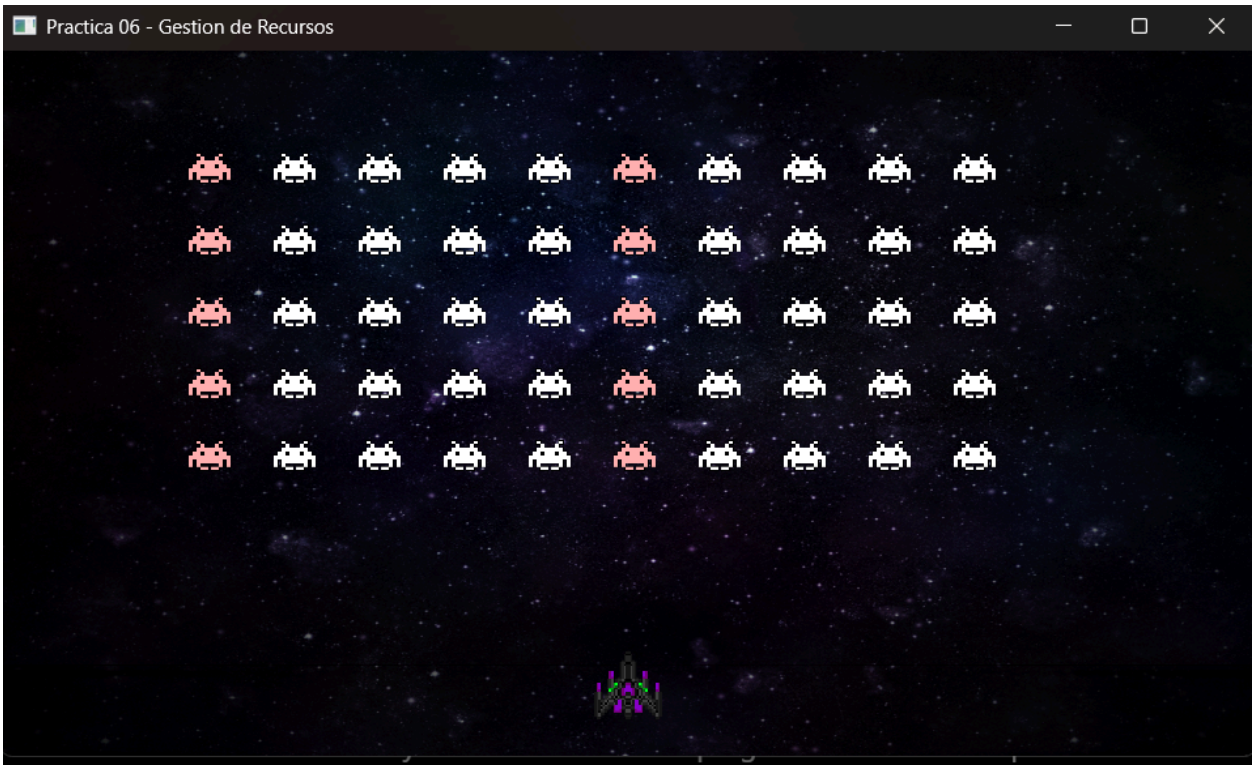
En un entorno con LeakSanitizer habilitado, el reporte puede identificar las reservas de memoria no liberadas y relacionarlas con la función `TriggerDeliberateLeak()` del archivo `ResourceDemoScene.cpp`.

Consideración del entorno: El proyecto fue desarrollado en Windows mediante MSYS2 UCRT64. Debido a las posibles incompatibilidades de AddressSanitizer en este entorno, el análisis con el sanitizador se considera opcional, conforme a las indicaciones de la práctica. No se cuenta todavía con una traza de ASan documentada.

Para el funcionamiento normal implementé un procedimiento de liberación en el que primero se eliminan las entidades que utilizan las texturas y posteriormente se ejecuta `ResourceManager::Clear()`, evitando conservar referencias a recursos destruidos.

6. Evidencias de funcionamiento

Captura 1. Escena de demostración



Captura 2. Métricas del patrón Flyweight

```
ellio@Piabonnie UCRT64 /c/motor de videojuegos C++/semana02/semana02/plantilla
$ ./build-ucrt64/proyecto01.exe
[ResourceManager] Textura cargada: assets/background.png (1150x864)
[ResourceManager] Textura cargada: assets/player.png (800x800)
[ResourceManager] Textura cargada: assets/enemy.png (650x650)
[ResourceManager] Reutilizando: assets/enemy.png
[ResourceDemoScene] Escena inicializada.
========================================
[Flyweight Metric] Entidades: 52
[Flyweight Metric] Texturas: 3
========================================
```

Captura 3. Liberación de recursos

```
ellio@Piabonnie UCRT64 /c/motor de videojuegos C++/semana02/semana02/plantilla
$ ./build-ucrt64/proyecto01.exe
========================================
[ResourceManager] Reutilizando: assets/enemy.png
========================================
[Flyweight Metric] Entidades: 60
[Flyweight Metric] Texturas: 3
========================================
[ResourceDemoScene] Reiniciando recursos...
[ResourceManager] Cache limpiada: 3 texturas.
[ResourceManager] Textura cargada: assets/background.png (1150x864)
[ResourceManager] Textura cargada: assets/player.png (800x800)
[ResourceManager] Textura cargada: assets/enemy.png (650x650)
[ResourceManager] Reutilizando: assets/enemy.png
[ResourceDemoScene] Escena inicializada.
========================================
[Flyweight Metric] Entidades: 52
[Flyweight Metric] Texturas: 3
========================================
[Leak Test] 50 superficies sin liberar.
[Leak Test] Cierra la aplicacion para revisar ASan.
[ResourceDemoScene] Cerrando escena...
[ResourceManager] Cache limpiada: 3 texturas.
[ResourceDemoScene] Recursos liberados.
[ResourceManager] Cache limpiada: 0 texturas.

ellio@Piabonnie UCRT64 /c/motor de videojuegos C++/semana02/semana02/plantilla
$
```

7. Conclusión

Con esta práctica comprendí la importancia de administrar correctamente los recursos gráficos dentro de un motor de videojuegos.

El patrón Flyweight me permitió reutilizar una misma textura entre múltiples entidades, evitando cargas repetidas y un consumo innecesario de memoria gráfica.

También aprendí a utilizar `std::unique_ptr` con destructores personalizados para administrar recursos de SDL3 mediante RAII, así como a diferenciar entre superficies almacenadas en RAM y texturas utilizadas por la GPU.

Finalmente, integré estos conceptos en la arquitectura de componentes de mi motor, manteniendo separadas las responsabilidades de carga, almacenamiento y renderizado.